

Como usar *o Cursor*

Na aula anterior você viu **o que** o Cursor é. Agora é a prática: instalar, abrir um projeto, pedir ao agente que explique o código, fazer uma primeira mudança, revisar o diff, planejar tarefas maiores, dar contexto com **@**, controlar o terminal e escrever as primeiras **rules** do projeto.

Instalar e entrar

Entender → mudar → revisar

Plan Mode

@ contexto

Run Modes

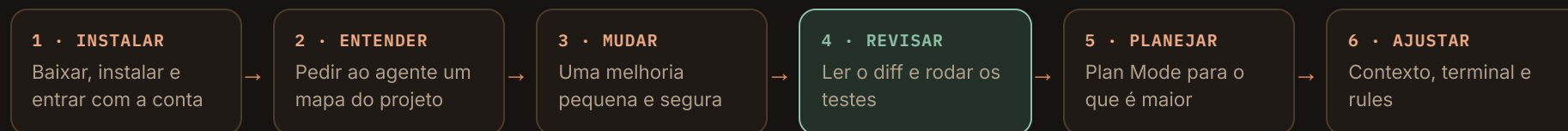
Rules .mdc

CLI

Formação Claude Code 2026 · Baseado em [cursor.com \(Docs: Quickstart, Prompting agents, Plan Mode, Run Modes, Browser, Rules, Worktrees, CLI; Help: Install, Your first project, @ mentions, Ignore files, Terminal, Keyboard shortcuts\)](https://cursor.com/docs), consultados em outubro de 2026 · Telas: reprodução ilustrativa

O fluxo *desta aula*

O próprio guia de início do Cursor propõe um caminho curto: da instalação até a primeira mudança útil. Esta aula segue esse roteiro e acrescenta o que você precisa para o dia a dia.



Se você já usa o Claude Code, vai reconhecer o ritmo: **explorar** → **planejar** → **executar** → **verificar**. Muda a interface, não o método.

Antes de começar

UMA CONTA

Crie grátis em cursor.com. O plano Hobby basta para seguir esta aula; os pedidos ao Agent é que são limitados.

UM PROJETO COM GIT

De preferência pequeno, ou uma cópia de teste. Assim, qualquer mudança do agente pode ser desfeita.

UMA TAREFA PEQUENA

Algo de baixo risco para a primeira mudança: um texto, um ajuste visual, um teste que falta.

Instalar *e entrar*

WINDOWS 10+

Baixe em cursor.com/download, rode o `.exe` e siga o assistente. Há versão para o sistema todo e só para o seu usuário (x64 e ARM64).

MACOS 12+

Abra o `.dmg` e arraste o Cursor para **Aplicativos**. Funciona em Apple Silicon e Intel.

LINUX

Prefira o repositório oficial (apt ou dnf): cria o ícone, atualiza sozinho e instala as ferramentas de linha de comando. O Applmage é a opção portátil.

```
# Debian/Ubuntu: chave, repositório e instalação
curl -fsSL https://downloads.cursor.com/keys/anysphere.asc | gpg --dearmor | sudo tee /etc/apt/keyrings/cursor.gpg > /dev/null
echo "deb [arch=amd64,arm64 signed-by=/etc/apt/keyrings/cursor.gpg] https://downloads.cursor.com/aptrepo stable main" | sudo tee
/etc/apt/sources.list.d/cursor.list > /dev/null
sudo apt update && sudo apt install cursor
# Fedora/RHEL: crie /etc/yum.repos.d/cursor.repo (baseurl https://downloads.cursor.com/yumrepo) e rode: sudo dnf install cursor
```

Ao abrir pela primeira vez, entre com a sua conta do Cursor (dá para criar uma grátis em cursor.com). O guia oficial diz que tudo leva menos de cinco minutos.

Abrir um projeto *e se situar*

1 **File** → **Open Folder** e escolha a pasta do projeto. Comece por um projeto pequeno ou uma cópia de teste.

2 Veio do VS Code? **Ctrl** + **Shift** + **J** → **General** → **Account** → **VS Code Import** traz extensões, temas, configurações e atalhos.

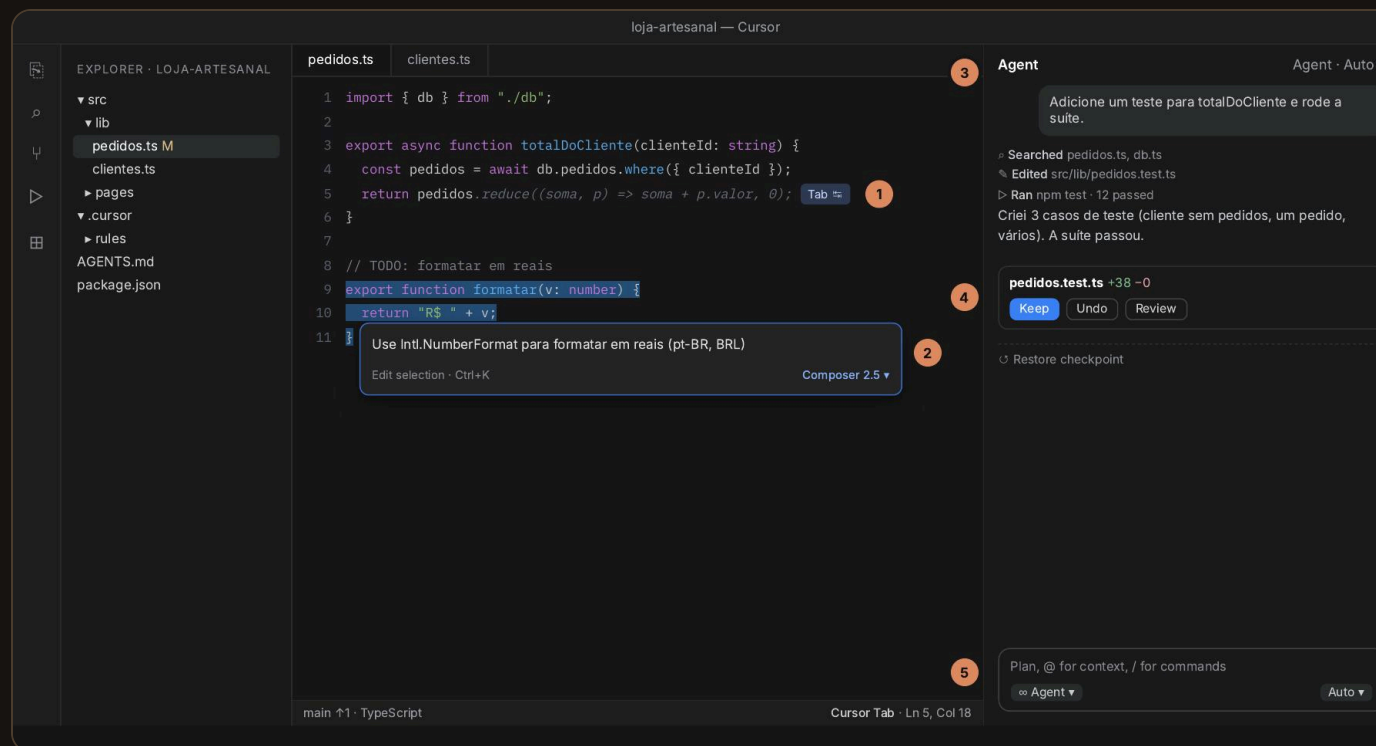
3 Abra o Agent com **Ctrl** + **I** (Mac: **Cmd** + **I**).

4 Confirme que o projeto está no **Git**. O agente grava direto no disco; o Git é a sua rede de segurança.

ONDE FICA	ATALHO
Configurações do Cursor	Ctrl + Shift + J
Paleta de comandos	Ctrl + Shift + P
Abrir arquivo pelo nome	Ctrl + P
Painel do Agent	Ctrl + I ou Ctrl + L

No Mac, troque **Ctrl** por **Cmd** em todos os atalhos desta aula.

As áreas *da tela*



O mesmo projeto de exemplo da aula 01 · reprodução ilustrativa (a interface é em inglês)

1 · TAB

Sugestões em cinza no editor.

2 · CTRL+K

Edição na seleção.

3 · AGENT

Conversa e passos executados.

4 · DIFF

Keep, Undo, Review.

5 · ENTRADA

Pedido, @, /, modo e modelo.

Peça um *mapa do projeto*

O primeiro pedido sugerido pela documentação não muda nada: pede ao agente que **explique o código**. Ele busca no repositório, lê os arquivos relevantes e resume como o projeto se organiza.

```
# Prompt sugerido pelo Quickstart (pode escrever em português)
Explique este projeto. Mostre os principais pontos de entrada, os módulos
mais importantes e o que eu preciso ler antes de fazer mudanças.
```

Quer garantia de que nada vai mudar? Troque para o modo Ask (`Shift` + `Tab`): ele é somente leitura.

Use a resposta como guia. Peça para detalhar um módulo, explicar uma função ou desenhar o fluxo de uma requisição. É o jeito mais rápido de se orientar num código desconhecido.

Uma primeira *mudança pequena*

Com o projeto entendido, peça sugestões e escolha uma. Boas primeiras tarefas têm **baixo risco**: um texto, um pequeno ajuste de interface.

Sugira três melhorias pequenas e seguras neste projeto.
Explique os prós e contras de cada uma e espere eu escolher.

Já sabe o que quer? Peça direto e **descreva o resultado esperado**, não só a ação.

ESCREVENDO VOCÊ MESMO

Comece a digitar e veja o **Tab**: **Tab** aceita, **Esc** recusa, **Ctrl** + **→** aceita palavra por palavra.

AJUSTE LOCALIZADO

Selecione o trecho, **Ctrl** + **K**, descreva a mudança e Enter. Para só perguntar, **Alt** + **Enter**.

VÁRIOS ARQUIVOS

Selecione e use **Ctrl** + **L**: o Agent abre com a seleção como contexto.

Revisar o diff *e verificar*

- 1 Acompanhe as edições aparecendo no **diff** enquanto o agente trabalha.
- 2 Leia cada mudança: **Keep** para manter, **Undo** para desfazer, **Review** para ver em detalhe.
- 3 Peça ao agente que rode as verificações que o projeto já tem: **testes, checagem de tipos, linter ou build**.
- 4 Gostou? Faça o commit. Não gostou? **Restore Checkpoint** na mensagem anterior volta os arquivos.

Rode os testes e o linter do projeto. Se algo falhar, explique a causa antes de corrigir.

Checkpoint não é commit. Ele desfaz o que o agente fez e fica só no seu computador. O histórico do projeto continua sendo o Git.

Errou o rumo no meio da tarefa? Mande uma correção com **Ctrl** + **Enter** (redireciona na hora) ou use **Stop**.

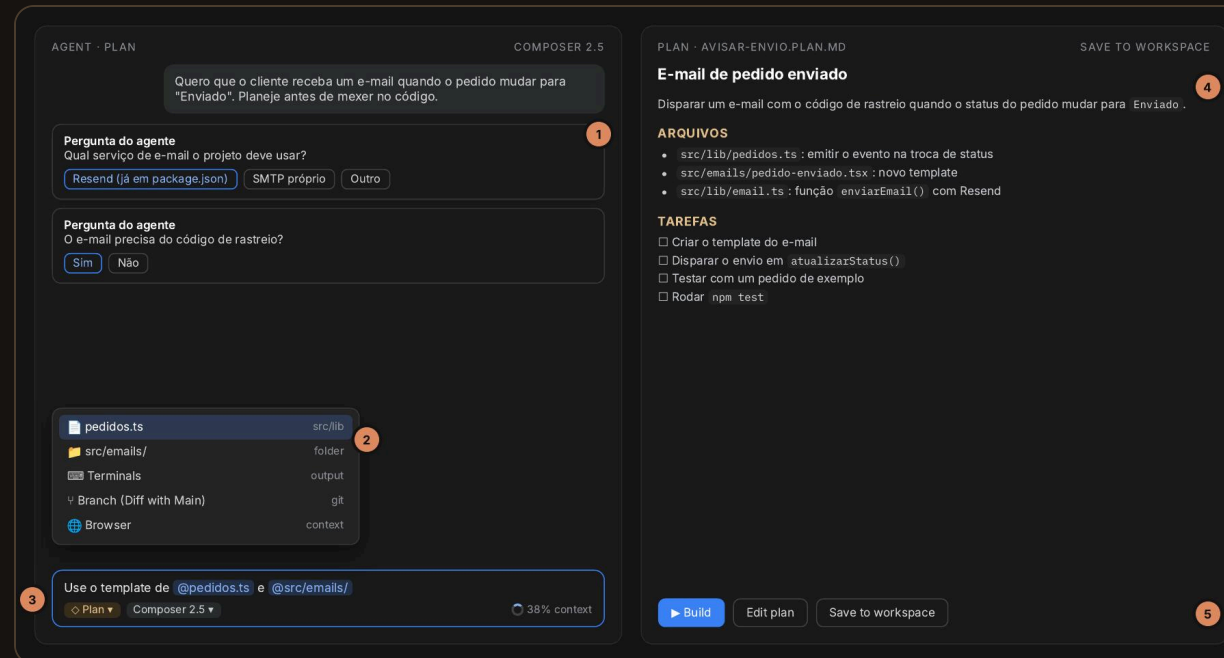
Apontar o caminho *com @*

O agente encontra os arquivos sozinho, mas quando **você sabe** o que importa, digite **@** na caixa de mensagem e anexe.

VOCÊ DIGITA	O QUE ENTRA NO CONTEXTO
@auth.ts · @src/components/	Arquivos e pastas
@Terminals	A saída do terminal
@Chats	Uma conversa anterior
@Commit · @Branch	Mudanças não commitadas ou o diff do branch contra a main
@Browser	O navegador embutido
@nome-da-rule	Uma rule aplicada manualmente

- **Imagens:** arraste ou cole (**Ctrl** + **V**) um print, um layout ou uma mensagem de erro.
- **Voz:** clique no microfone e dite o pedido; revise a transcrição antes de enviar.
- **Anel de contexto:** ao lado da caixa de mensagem, mostra quanto da janela do modelo já foi usada. Clique para ver por categoria (rules, ferramentas, MCP, conversa...).
- **Janela cheia:** o Cursor resume as partes antigas da conversa. Para uma tarefa nova, prefira uma conversa nova.

Plan Mode para *o que é maior*



1 perguntas do agente · 2 menu @ · 3 modo Plan eanel de contexto · 4 o plano editável · 5 Build · reprodução ilustrativa

ATIVE

Shift + **Tab** até o modo **Plan**.
O Cursor também sugere sozinho
em pedidos complexos.

RESPONDA

O agente pesquisa o código e faz
perguntas para fechar os
requisitos.

EDITE

O plano sai em Markdown. Ajuste
pelo chat ou direto no arquivo;
Save to workspace guarda no
projeto.

CONSTRUA

Clique em **Build**. Ficou diferente
do esperado? Desfaça, refine o
plano e rode de novo.

Escolher *o modelo certo*

COMO	ONDE
Trocar nesta conversa	Seletor no topo da caixa de mensagem
Alternar rapidamente	Ctrl + /
Definir o padrão	Cursor Settings → Models
Deixar o Cursor decidir	Auto

A troca vale dali para a frente na mesma conversa. Dá para explorar com um modelo rápido e trocar por um mais capaz na hora de implementar.

MODELOS RÁPIDOS
Edições simples e tarefas rotineiras. Ex.: Composer 2.5, que também tem mais uso incluído nos planos pagos.

MODELOS MAIS CAPAZES
Raciocínio complexo e refatorações em vários arquivos. Ex.: Claude Opus 5.5, GPT-5.6 Sol.

Lembre da aula 01: o modelo escolhido define **quanto do uso incluído** cada pedido consome.

Quanto o agente *pode rodar sozinho*

O agente roda comandos (instalar dependências, testes, build) conforme o **Run Mode**, em **Cursor Settings** → **Agents** → **Approvals & Execution**.

RUN MODE	O QUE RODA SEM PERGUNTAR	PARA QUEM
Auto-review (padrão)	Comandos da lista de permitidos; os outros rodam numa sandbox quando possível; o resto passa por um classificador que libera ou bloqueia	A maioria das pessoas
Allowlist	Só o que está na sua lista (com sandbox opcional)	Quem quer controle fino
Run Everything	Tudo, sem aprovação	Ambientes descartáveis

```
# .cursor/permissions.json (no projeto) ou
~/ .cursor/permissions.json
{ "autoRun": { "block_instructions": [
  "Todo comando de deploy deve pedir aprovação." ] } }
```

O classificador do Auto-review **não é uma barreira de segurança**: pode errar. Dica da documentação: peça ao próprio agente "quero aprovar todo comando X" e ele ajusta o `permissions.json`.

O agente *testa no navegador*

Sem instalar nada, o Agent controla um navegador para testar o app, conferir mudanças visuais e investigar erros.

- **Navega, clica, digita e rola** a página.
- **Tira capturas de tela** e "enxerga" o resultado.
- **Lê o console** e o **tráfego de rede** para achar erros.
- Detecta o **servidor de desenvolvimento** já rodando e usa a porta certa.
- Cookies e login **persistem por projeto**, isolados entre projetos.

Exemplo da documentação (acessibilidade)

@Browser Verifique o contraste das cores, o HTML semântico e os rótulos ARIA, teste a navegação pelo teclado e aponte imagens sem texto alternativo.

Combine com o fluxo da aula: o agente **muda** o código, **abre** a página e **confere** o resultado antes de você revisar.

Suas primeiras *rules*

Project rules ficam em `.cursor/rules/`, em arquivos `.mdc` (um `.md` comum ali é ignorado). O cabeçalho define **quando** a rule entra:

TIPO	CABEÇALHO
Always Apply	<code>alwaysApply: true</code>
Por arquivo	<code>globs: src/**/*.tsx</code>
Inteligente	<code>description: ...</code> (o agente decide)
Manual	sem os três: só com <code>@nome</code>

```
# .cursor/rules/componentes.mdc
---
globs: src/components/**/*.tsx
alwaysApply: false
---
- Use exports nomeados, não default
- Mantenha componentes com menos de 200 linhas
- Prefira composição a repassar props por várias camadas
```

Prefere simplicidade? Um `AGENTS.md` na raiz também vale como rule. E o CLI do Cursor lê até um `CLAUDE.md` existente.

O que o agente *não deve ler*

Crie um `.cursorignore` na raiz com os padrões a excluir do contexto da IA:

```
node_modules/  
dist/  
*.min.js  
.env*
```

- ✓ O Cursor já ignora `.env`, `.git/` e arquivos de lock
- ✓ Respeita automaticamente o `.gitignore`
- ✓ Use o `.cursorignore` para o que vai além dele

POR QUE IGNORAR

Arquivos gerados e grandes, segredos e credenciais, binários e código de terceiros só trazem ruído ou risco.

Limite importante: comandos de terminal e ferramentas MCP rodam fora desse controle e ainda podem ler arquivos ignorados. Segredo de verdade não fica no repositório.

Ligue o **Privacy Mode** (Settings → General) para que seu código não seja usado em treinamento.

Mais de um agente *ao mesmo tempo*

WORKTREES

Cada agente trabalha numa cópia isolada do Git, sem conflito com a sua pasta principal. A interface de worktrees fica na **Agents Window**; no editor, use os comandos de worktree.

PREPARAR A CÓPIA

O arquivo `.cursor/worktrees.json` diz o que rodar ao criar cada worktree, como `npm ci` e copiar o `.env`.

MANDAR PARA A NUVEM

Na Agents Window, `/in-cloud` passa a tarefa a um agente na nuvem. No CLI, comece a mensagem com `&`.

```
# .cursor/worktrees.json (exemplo da documentação para Node.js)
{ "setup-worktree": [ "npm ci", "cp $ROOT_WORKTREE_PATH/.env .env" ] }
```

A documentação não recomenda criar links simbólicos para dependências dentro do worktree. Use um gerenciador rápido, como `pnpm`, `bun` ou `uv`.

O Cursor *no terminal*

```
# Instalar (macOS, Linux, WSL)
curl https://cursor.com/install -fsS | bash
# Instalar (Windows PowerShell)
irm 'https://cursor.com/install?win32=true' | iex

# Sessão interativa
agent
# Já com um pedido
agent "refatore o módulo de autenticação para usar JWT"
# Começar planejando ou só perguntando
agent --plan
agent --mode=ask
```

NO CLI	COMO
Alternar Agent, Plan, Ask	<code>Shift</code> + <code>Tab</code> ou <code>/plan</code> , <code>/ask</code>
Revisar as mudanças	<code>Ctrl</code> + <code>R</code>
Anexar arquivos	<code>@</code>
Liberar contexto	<code>/summarize</code>
Nova linha no pedido	<code>Shift</code> + <code>Enter</code> ou <code>Ctrl</code> + <code>J</code>
Mandar para a nuvem	Comece com <code>&</code>
Sair	<code>Ctrl</code> + <code>D</code> duas vezes

O CLI usa as mesmas rules, o mesmo `mcp.json` e os mesmos modos do editor. Lê `AGENTS.md` e `CLAUDE.md` na raiz.

Atalhos *para ter à mão*

AÇÃO	WINDOWS / LINUX	MAC
Abrir/fechar o painel do Agent	Ctrl + I ou Ctrl + L	Cmd + I ou Cmd + L
Edição inline	Ctrl + K	Cmd + K
Alternar entre os modos	Shift + Tab	Shift + Tab
Menu de modos	Ctrl + .	Cmd + .
Alternar entre modelos	Ctrl + /	Cmd + /
Aceitar sugestão do Tab	Tab	Tab
Configurações do Cursor	Ctrl + Shift + J	Cmd + Shift + J
Personalizar atalhos	Ctrl + R , depois Ctrl + S	Cmd + R , depois Cmd + S

Os demais atalhos são os mesmos do VS Code. Se você importou seus atalhos personalizados, eles continuam valendo.

Hábitos que *fazem diferença*

- ✓ Uma conversa por tarefa; ao mudar de assunto, abra outra
- ✓ Descreva o **resultado esperado**, não só a ação
- ✓ Use @ quando souber os arquivos; deixe o agente buscar quando não souber
- ✓ Plan Mode para tudo que toca vários arquivos
- ✓ Peça testes, linter e build antes de aceitar
- ✓ Commit pequeno depois de cada mudança aprovada
- ✓ Rules curtas e específicas, versionadas com o código
- ✓ Segredos fora do repositório e do contexto
- ✓ Run Mode em Auto-review; Run Everything só em ambiente descartável
- ✓ Fique de olho no anel de contexto e no consumo do plano

Quando o agente erra feio: em vez de remendar com vários pedidos, desfaça (checkpoint), refine o pedido ou o plano e rode de novo. A documentação diz que isso costuma ser mais rápido e dá um resultado mais limpo.

Teste o que *ficou*

1. Em que pasta e extensão ficam as Project Rules?

A) `.cursor/rules`, arquivos
`.mdc`

B) `.cursor/rules`, arquivos
`.txt`

C) Na raiz, arquivos `.json`

D) Só nas configurações

A. Um `.md` comum ali é ignorado.

2. Qual é o Run Mode padrão?

A) Run Everything

B) Ask Every Time

C) Auto-review

D) Allowlist sem sandbox

C. Lista, sandbox e classificador.

3. Como incluir a saída do terminal no pedido?

A) Copiar e colar sempre

B) `@Terminals`

C) `/terminal`

D) Não é possível

B. O `@` anexa contexto.

4. O que fazer quando o resultado do Plan Mode não ficou como esperado?

A) Remendar com dez pedidos

B) Apagar o projeto

C) Desfazer, refinar o plano e construir de novo

D) Trocar de editor

C. É o conselho da própria documentação.

○ RECAPITULANDO

Entender, mudar, *revisar, planejar*

Instale, abra o projeto e comece pedindo um **mapa do código**. Faça uma mudança pequena, **revise o diff** e peça os testes antes de aceitar. Para o que é maior, use o **Plan Mode**. Dê contexto com **@**, controle o terminal com os **Run Modes**, proteja arquivos com o **.cursorignore** e registre as convenções do projeto em **rules**. No terminal, o comando `agent` leva tudo isso para o CLI.

Ctrl+I · Ctrl+K · Shift+Tab

Keep · Undo · Checkpoint

Plan → Build

@ arquivos · @Terminals · @Browser

.cursor/rules/*.mdc

agent no terminal

Próxima aula → Principais Funcionalidades

Fontes: [cursor.com](https://cursor.com/docs) (Docs: Quickstart, Prompting agents, Plan Mode, Run Modes, Browser, Rules, Worktrees, CLI Overview e Using Agent in CLI; Help: Download and install, Your first project, @ mentions and context, Ignore files, Terminal integration, Keyboard shortcuts, Migrate from VS Code), consultados em outubro de 2026. Produto em evolução rápida: menus e nomes podem mudar.